8/4 4회차 스터디



🛠️ Nginx(1)

https://github.com/likelion-server-study/WEEK4

▼ 예습해오기

다음 주 4주차 스터디 안내드립니다!

이번 주에는 PetClinic·PostgreSQL 앞에 Nginx를 추가해 아래 구조를 직접 구성해봅니다.

사용자 → Nginx → PetClinic → PostgreSQL

Nginx를 통해 앱 직접 접근을 막고, 요청 전달·정적 파일 캐시·요청 제한·로그 확인·앱 중지 시 502 오류와 복구를 확인해 보려고 합니다.

사전 준비 자료는 금일 저녁까지 완성해서 올리겠습니다. (완성했으면 완성했다고 말씀드리겠습니다..!) 다음 모임 전에 아래 자료 보고 와주시면 좋겠습니다.

- Nginx 영상

  → Nginx를 왜 쓰는지와 웹 서버·리버스 프록시·캐시·로드밸런서 역할을 가볍게 이해하는 영상입니다.

- 그림으로 이해하는 서버 구조와 기술 — 112p ~ 119p
  - 4.1 웹 서버
  - 4.2 웹 서버 통신
  - 4.3 웹 애플리케이션 서버
  - 4.4 3계층 클라이언트 서버 시스템
    → Nginx, PetClinic, PostgreSQL이 각각 어떤 역할인지 이해하는 데 도움이

됩니다.

- 이것이 리눅스다 — 823p 프록시 개념

  → 이번에 사용하는 리버스 프록시가 요청을 어떻게 전달하는지 이해하기 위한 내용입니다.

▼ 4회차에서 헷갈리기 쉬운 개념 정리

▼ `localhost`와 포트

`localhost`는 현재 사용하는 내 컴퓨터를 뜻합니다. 하지만 컴퓨터 안에는 여러 프로그램이 실행될 수 있으므로, 어느 프로그램이 요청을 받을지 포트 번호로 구분합니다.

```
localhost:8080
→ 내 컴퓨터의 8080번 포트를 받고 있는 프로그램
→ 이번 실습에서는 Nginx
```

이번 구성에서 `localhost:8080`은 Docker가 Nginx 컨테이너 내부의 `80` 포트로 전달합니다.

```
localhost:8080
→ Nginx 컨테이너:80
```

▼ 호스트 포트와 컨테이너 내부 포트

```
ports: ["8080:80"]
```

왼쪽 `8080`은 내 컴퓨터에서 접근할 포트이고, 오른쪽 `80`은 Nginx 컨테이너 내부 포트입니다.

```
내 컴퓨터:8080
→ Nginx 컨테이너:80
```

반면 `app:8080`, `postgres:5432`는 Docker 내부 네트워크에서 컨테이너끼리 사용하는 주소입니다. 호스트에 `ports:`로 공개하지 않았으므로 브라우저가 직접 접근할 수 없습니다.

▼ Docker 서비스 이름과 주소

Compose의 서비스 이름은 Docker 내부 DNS 이름이 됩니다.

```
app
→ app 컨테이너의 내부 주소 이름

postgres
→ PostgreSQL 컨테이너의 내부 주소 이름
```

그래서 Nginx는 `app:8080`으로 PetClinic에 전달하고, PetClinic은 `postgres:5432`로 DB에 연결합니다.

▼ `server {}`와 `location {}`

Nginx의 `server {}`는 뒷단 app 서버가 아닙니다. Nginx가 받은 요청을 어떤 웹사이트 규칙으로 처리할지 묶는 가상 웹 서버 설정입니다.

```
server {
  listen 80;
}
```

뜻은 "Nginx가 80번 포트로 받은 요청을 이 규칙으로 처리한다"입니다.

`location {}`은 URL 경로별 처리 규칙입니다.

```
location / {
  proxy_pass http://app:8080;
}
```

뜻은 "`/`로 시작하는 요청은 app 컨테이너의 8080 포트로 전달한다"입니다.

```
server {}
→ 어떤 포트·도메인 요청을 처리할지

location {}
→ URL 경로별로 어떻게 처리할지

proxy_pass
→ 실제 뒷단 app 주소
```

▼ `00-observability.conf` 와 `default.conf`

두 파일은 Nginx가 함께 읽는 설정 파일입니다. Nginx가 두 개로 실행되는 것이 아닙니다.

```
00-observability.conf
→ 공통 기준 정의
→ 로그 형식, 캐시 영역, 요청 제한 기준

default.conf
→ 실제 요청 처리
→ 프록시, 캐시 적용, 차단, 오류 페이지
```

`limit_req_zone`은 "요청 제한 기준을 만들기"만 합니다. 실제 차단은 `default.conf`의 `location /` 안에서 `limit_req`를 적용해야 일어납니다.

▼ Docker 볼륨과 Nginx 캐시 이름

```
nginx-cache:/var/cache/nginx
```

`nginx-cache`는 Docker가 관리하는 실제 저장 공간입니다.

```
proxy_cache_path /var/cache/nginx ... keys_zone=petclinic_cache:10m;
```

`petclinic_cache`는 Nginx가 캐시 목록을 관리할 때 쓰는 이름입니다.

```
nginx-cache
→ 캐시 파일을 저장할 Docker 공간

/var/cache/nginx
→ 컨테이너 안에서 그 공간이 연결된 폴더

petclinic_cache
→ Nginx 설정에서 캐시 저장소를 부르는 이름
```

▼ `BYPASS`, `MISS`, `HIT`

이 값들은 Nginx가 제공하는 캐시 상태입니다.

```
BYPASS
→ 캐시를 일부러 사용하지 않음
→ app까지 전달
```

```
MISS
→ 캐시에 없음
→ app에서 받아 캐시에 저장

HIT
→ 캐시에 있음
→ Nginx가 직접 응답, app에 전달하지 않음
```

▼ `curl -w`와 Nginx 로그 형식의 차이

`curl -w`는 요청을 보낸 사용자 쪽에서 터미널에 시간·상태를 출력하는 형식입니다.

```
-w 'total=%{time_total}s'
```

Nginx의 `log_format`은 Nginx 서버가 요청 처리 결과를 접근 로그에 남기는 형식입니다.

```
log_format study ...
```

```
curl -w
→ 요청한 쪽에서 본 결과

Nginx log_format
→ 서버가 기록한 처리 과정
```

▼ `o /dev/null`

`/dev/null`은 일반 폴더가 아니라 받은 데이터를 버리는 리눅스 특수 장치입니다.

```
-o /dev/null
```

이미지 본문은 화면에 출력하지 않고 버리며, 상태 코드·응답 헤더·시간만 확인할 때 사용합니다.

▼ Nginx 설정 반영

```
설정 수정
→ nginx -t
```

```
→ 성공한 경우만 nginx -s reload
```

`nginx -t`가 실패하면 reload하지 않습니다. Nginx는 기존에 정상 동작하던 설정을 계속 사용합니다.

같은 범위에 `client_max_body_size`나 `proxy_intercept_errors`를 두 번 작성하면 duplicate 오류가 발생할 수 있습니다.

▼ 로그의 역할

```
curl
→ 사용자가 실제로 받은 상태 코드·응답 헤더

Nginx access log
→ 상태 코드, 캐시 상태, app 전달 여부

Nginx error log
→ app 연결 실패 등 오류 원인

Tomcat access log
→ 요청이 PetClinic까지 실제 도달했는지

PostgreSQL 로그
→ DB 기동·연결 준비 상태
```

Nginx 로그에는 있는데 Tomcat 로그에는 없다면, Nginx가 app 앞에서 요청을 캐시 처리하거나 차단한 경우일 수 있습니다.

▼ 5회차 미리보기

5주차에는 4주차에서 만든 Nginx 앞단 구조를 유지하면서, Nginx 뒤에 PetClinic 앱 컨테이너를 두 개 배치합니다.

지금까지 Nginx는 모든 요청을 하나의 `app:8080`으로 전달했습니다. 이번 주차에는 `app1:8080`, `app2:8080`을 하나의 앱 서버 묶음으로 등록하고, Nginx가 요청을 나누어 전달하도록 구성합니다.

```
사용자
  ↓ localhost:8080
Nginx
  ├─ app1:8080
  └─ app2:8080
         ↓
```

```
    postgres:5432
    PostgreSQL
```

두 PetClinic은 별개의 컨테이너로 실행되지만 같은 PostgreSQL 데이터를 사용합니다. 사용자는 어떤 앱이 요청을 처리하는지 신경 쓰지 않고 이전과 동일하게 Nginx로만 접속합니다.

5주차에 확인할 문제

- 같은 애플리케이션을 여러 컨테이너로 실행할 수 있는가

  → 기존 단일 `app`을 `app1`, `app2`로 나누고 같은 PetClinic 이미지로 실행합니다.

  → 두 앱은 호스트 포트를 공개하지 않고 Compose 내부의 `8080` 포트만 사용합니다.

- 여러 앱이 하나의 PostgreSQL을 함께 사용할 수 있는가

  → `app1`, `app2` 모두 `postgres:5432`에 연결합니다.

  → 어느 앱에서 요청을 처리하더라도 같은 PetClinic 데이터가 조회되는지 확인합니다.

- 어느 앱이 요청을 처리했는지 구분할 수 있는가

  → app1·app2의 Tomcat access log 저장 경로를 분리합니다.

  → Nginx의 upstream 기록과 각 앱의 Tomcat 로그를 함께 확인합니다.

- Nginx는 요청을 여러 앱에 어떻게 나누는가

  → Nginx의 `upstream`에 `app1:8080`, `app2:8080`을 등록합니다.

  → 기본 라운드 로빈 방식으로 요청이 두 앱에 분산되는지 확인합니다.

- 앱 하나가 멈추면 서비스도 중단되는가

  → Nginx에 장애 우회 설정을 적용합니다.

  → 앱 하나를 중지해도 실행 중인 다른 앱이 요청을 처리하여 `200 OK`가 유지되는지 확인합니다.

- 모든 앱이 멈추면 어떻게 되는가

  → app1·app2를 모두 중지합니다.

  → Nginx는 실행 중이지만 전달할 앱이 없으므로 `502 Bad Gateway`를 반환하는지 확인합니다.

  → 4주차에서 만든 장애 안내 화면이 표시되더라도 HTTP 상태는 `502`로 유지되는지 확인합니다.

다음 주에 사용하는 구성

구성 요소	역할
Docker Compose	Nginx·app1·app2·PostgreSQL을 하나의 구성으로 관리
Nginx	외부 요청을 받고 여러 앱 중 한 곳으로 전달
`upstream`	Nginx가 요청을 전달할 앱 서버 목록
Round Robin	요청을 등록된 앱에 순서대로 분산하는 기본 방식
app1·app2	같은 PetClinic을 각각 실행하는 앱 컨테이너
PostgreSQL	두 PetClinic이 함께 사용하는 데이터 저장소
Tomcat access log	어느 앱이 요청을 처리했는지 확인하는 기록
장애 우회 설정	앱 연결 실패 시 다른 앱으로 요청을 다시 전달
502 오류 페이지	모든 앱이 중지되었을 때 안내 화면 제공

▼ 간단한 진행 흐름

1. 단일 앱을 두 개의 앱으로 구성

기존 `app` 구성을 기준으로 `app1`, `app2`를 만듭니다.

```
app1
- PetClinic 실행
- 내부 포트 8080
- DB 주소 postgres:5432

app2
- PetClinic 실행
- 내부 포트 8080
- DB 주소 postgres:5432
```

두 앱에는 `ports:`를 설정하지 않습니다. 외부 사용자는 계속 Nginx의 호스트 `8080` 포트로만 접속합니다.

2. 두 앱을 하나의 PostgreSQL에 연결

app1·app2의 DB 주소를 모두 `postgres:5432`로 설정합니다.

```
app1 ─┐
      ├─ postgres:5432
app2 ─┘
```

두 앱에서 같은 데이터를 조회하고 변경할 수 있는지 확인합니다.

> **주의:** 4주차까지 사용한 PostgreSQL Named Volume을 그대로 유지합니다. 이미 테이블이 만들어진 DB에서는 두 앱이 시작하면서 초기화 SQL을 다시 실행하지 않도록 설정합니다. 이는 일반적인 DB 동시 접근을 막는 설정이 아니라, 시작 시 테이블 생성 작업이 중복되는 것을 방지하기 위한 설정입니다.

3. app1·app2의 로그 저장 경로 분리

두 앱이 같은 로그 파일에 기록하지 않도록 Tomcat access log 경로를 분리합니다.

```
app1 → logs/tomcat/app1
app2 → logs/tomcat/app2
```

각 앱의 로그를 따로 확인하면 어느 앱이 요청을 처리했는지 알 수 있습니다.

4. Nginx upstream 설정

Nginx에 두 앱을 하나의 서버 묶음으로 등록합니다.

```
upstream petclinic_backend {
    server app1:8080 max_fails=1 fail_timeout=10s;
    server app2:8080 max_fails=1 fail_timeout=10s;
}
```

기존 전달 주소를 다음처럼 변경합니다.

```
proxy_pass http://petclinic_backend;
```

이제 Nginx는 `app1` 하나가 아니라 `petclinic_backend`에 등록된 앱들로 요청을 나누어 전달합니다.

5. 장애 우회 설정

앱 연결에 실패했을 때 다른 앱을 시도하도록 설정합니다.

```
proxy_next_upstream error timeout http_502 http_503 http_504;
```

```
proxy_next_upstream_tries 2;
```

`max_fails`와 `fail_timeout`은 연결 실패가 발생한 앱을 일정 시간 동안 피하기 위한 기준입니다. `proxy_next_upstream`은 현재 앱으로 요청을 전달하지 못했을 때 다른 앱을 시도할 조건을 정합니다.

이 방식은 Nginx가 앱 상태를 미리 검사하는 능동 헬스체크가 아닙니다. 실제 요청을 전달하는 중 연결 실패가 발생했을 때 이를 확인하고 다른 앱으로 우회하는 방식입니다.

6. 전체 컨테이너 실행

```
docker compose config
docker compose up -d
docker compose ps
```

`nginx`, `app1`, `app2`, `postgres`가 모두 실행 중인지 확인합니다.

호스트 포트는 Nginx의 `8080`만 공개되어야 합니다.

```
nginx     → 0.0.0.0:8080->80/tcp
app1      → 8080/tcp
app2      → 8080/tcp
postgres  → 5432/tcp
```

7. 요청이 두 앱에 분산되는지 확인

캐시되지 않는 `/owners` 경로로 여러 요청을 보냅니다.

```
for i in $(seq 1 10); do
  curl -s -o /dev/null -w "%{http_code}\n" \
    "http://localhost:8080/owners?study=week5-$i"
done
```

Nginx 로그와 각 앱의 Tomcat 로그를 비교합니다.

```
docker compose logs --since 5m nginx
grep -R "study=week5-" logs/tomcat/app1
grep -R "study=week5-" logs/tomcat/app2
```

app1·app2 양쪽 로그에 요청이 나뉘어 기록되고 응답이 `200`이면 정상입니다.

> **주의:** 4주차의 정적 파일 캐시 설정은 그대로 유지합니다. 다만 이미지 요청은 Nginx 캐시에서 끝날 수 있으므로 로드밸런싱 확인에는 사용하지 않습니다. `/owners`처럼 PetClinic까지 전달되는 동적 요청으로 검사합니다.

8. 앱 하나를 중지하고 장애 우회 확인

먼저 app1만 중지합니다.

```
docker compose stop app1
curl -i "http://localhost:8080/owners?study=app1-down"
```

app2가 실행 중이므로 최종 응답은 `200 OK`여야 합니다.

다음 기록을 함께 확인합니다.

```
Nginx error log
→ app1 연결 실패 여부

Nginx access log
→ 최종 상태코드와 실제 upstream 주소

app2 Tomcat access log
→ 요청이 app2까지 도달했는지
```

확인 후 app1을 다시 시작합니다.

```
docker compose start app1
```

같은 방법으로 app2를 중지했을 때 app1이 요청을 처리하는지도 확인합니다.

9. 모든 앱을 중지하고 502 확인

```
docker compose stop app1 app2
curl -i "http://localhost:8080/owners?study=all-down"
```

전달할 앱이 하나도 없으므로 다음 결과가 나와야 합니다.

```
HTTP/1.1 502 Bad Gateway
```

브라우저에는 4주차에서 만든 장애 안내 화면이 표시될 수 있지만 HTTP 상태는 `502`로 유지되어야 합니다.

10. 앱을 다시 시작하고 복구 확인

```
docker compose start app1 app2
docker compose ps
curl -i "http://localhost:8080/owners?study=recovered"
```

두 앱이 다시 실행되고 응답이 `200 OK`로 복구되는지 확인합니다.

성공 기준

- 같은 PetClinic을 실행하는 app1·app2 컨테이너가 함께 실행된다.
- app1·app2는 호스트 포트를 공개하지 않는다.
- 두 앱은 하나의 `postgres:5432`와 기존 Named Volume을 함께 사용한다.
- app1·app2의 Tomcat access log가 서로 다른 경로에 저장된다.
- Nginx가 `upstream`을 통해 요청을 두 앱에 나누어 전달한다.
- 캐시되지 않는 `/owners` 요청이 두 앱의 로그에 나뉘어 기록된다.
- 앱 하나를 중지해도 다른 앱이 요청을 처리하여 `200 OK`가 유지된다.
- 두 앱을 모두 중지하면 안내 화면과 함께 `502 Bad Gateway`가 반환된다.
- 두 앱을 다시 시작하면 서비스가 `200 OK`로 복구된다.